

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №1
По предмету: “Вычислительная математика”
Вариант № 13**

Выполнил: Федоров Д. А.

Р3206

Проверил: Зинченко А. А.

г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ 3

1. 3

2. 3

3. 3

4. 4

Исходный код программы:..... 4

Результаты работы программы:..... 5

Заключение:..... 7

ЗАДАНИЕ:

Лабораторная работа №3. Численное интегрирование

Цель работы: найти приближенное значение определенного интеграла с требуемой точностью различными численными методами.

№ варианта определяется как номер в списке группы согласно ИСУ.

Лабораторная работа состоит из двух частей: вычислительной и программной.

Обязательное задание (до 80 баллов)

Исходные данные:

1. Пользователь выбирает функцию, интеграл которой требуется вычислить (3-5 функций) из тех, которые предлагает программа.
2. Пределы интегрирования задаются пользователем.
3. Точность вычисления задается пользователем.
4. Начальное значение числа разбиения интервала интегрирования: $n=4$.
5. Ввод исходных данных осуществляется с клавиатуры.

Программная реализация задачи:

1. Реализовать в программе методы по выбору пользователя:
 - Метод прямоугольников (3 модификации: левые, правые, средние)
 - Метод трапеций
 - Метод Симпсона
2. Методы должны быть оформлены в виде отдельной(ого) функции/класса.
3. Вычисление значений функции оформить в виде отдельной(ого) функции/класса.
4. Для оценки погрешности и завершения вычислительного процесса использовать правило Рунге.
5. Предусмотреть вывод результатов: значение интеграла, число разбиения интервала интегрирования для достижения требуемой точности.

Вычислительная реализация задачи:

1. Вычислить интеграл, приведенный в таблице 1, точно.
2. Вычислить интеграл по формуле Ньютона – Котеса при $n = 6$.
3. Вычислить интеграл по формулам средних прямоугольников, трапеций и Симпсона при $n = 10$.
4. Сравнить результаты с точным значением интеграла.
5. Определить относительную погрешность вычислений для каждого метода.
6. В отчете отразить последовательные вычисления.

Необязательное задание (до 20 баллов)

1. Установить сходимость рассматриваемых несобственных интегралов 2 рода (2-3 функции). Если интеграл - расходящийся, выводить сообщение: «Интеграл не существует».
2. Если интеграл сходящийся, реализовать в программе вычисление несобственных интегралов 2 рода (заданными численными методами).
3. Рассмотреть случаи, когда подынтегральная функция терпит бесконечный разрыв: 1) в точке a , 2) в точке b , 3) на отрезке интегрирования

Оформить отчет, который должен содержать:

1. Титульный лист.
2. Цель лабораторной работы.
3. Порядок выполнения работы.
4. Рабочие формулы методов.
5. Листинг программы.
6. Результаты выполнения программы.
7. Вычисление заданного интеграла.
8. Выводы

Вычислительная часть:

Вариант 11

$$\int_1^3 (2x^3 - 3x^2 + 7x - 10) dx$$

1. Точно:

$$\begin{aligned}\int_1^3 (2x^3 - 3x^2 + 7x - 10) dx &= \left(\frac{1}{2}x^4 - x^3 + \frac{7}{2}x^2 - 10x \right) \Big|_1^3 \\ &= \left(\frac{1}{2} \cdot 81 - 27 + \frac{7}{2} \cdot 9 - 30 \right) - \left(\frac{1}{2} - 1 + \frac{7}{2} - 10 \right) = 22\end{aligned}$$

2. Формула Ньютона – Котеса ($n = 6$):

$$\begin{aligned}\int_a^b f(x) dx &\approx \sum_{i=0}^n f(x_i) c_n^i = c_6^0 \cdot f(x_0) + c_6^1 \cdot f(x_1) + c_6^2 \cdot f(x_2) + c_6^3 \cdot f(x_3) + c_6^4 \cdot f(x_4) \\ &\quad + c_6^5 \cdot f(x_5) + c_6^6 \cdot f(x_6)\end{aligned}$$

$$x_i = a + i \cdot h, \text{ где } h = \frac{b-a}{n}, i = 1, 2, \dots, n$$

$$x_0 = 1, x_1 = \frac{4}{3}, x_2 = \frac{5}{3}, x_3 = 2, x_4 = \frac{7}{3}, x_5 = \frac{8}{3}, x_6 = 3$$

$$\begin{aligned}\sum_{i=0}^6 f(x_i) c_6^i &= \left(\frac{82}{840} \right) \cdot (-4) + \left(\frac{432}{840} \right) \cdot \left(-\frac{34}{27} \right) + \left(\frac{54}{840} \right) \cdot \left(\frac{70}{27} \right) + \left(\frac{544}{840} \right) \cdot 8 + \left(\frac{54}{840} \right) \\ &\quad \cdot \left(\frac{416}{27} \right) + \left(\frac{432}{840} \right) \cdot \left(\frac{682}{27} \right) + \left(\frac{82}{840} \right) \cdot 38 = \frac{18480}{840} = 22\end{aligned}$$

3. Формула средних прямоугольников ($n = 10$):

$$x_{i-\frac{1}{2}} = x_{i-1} + \frac{h_i}{2}, \text{ где } h = \frac{b-a}{n}, i = 1, 2, \dots, n$$

i	$x_{i-\frac{1}{2}}$	f(x)
1	1,1	-3,27
2	1,3	-1,57
3	1,5	0,5
4	1,7	3,06
5	1,9	6,19
6	2,1	10,01
7	2,3	14,61
8	2,5	20,08
9	2,7	26,54
10	2,9	34,07

$$\int_a^b f(x)dx \approx \sum_{i=0}^n h_i f\left(x_{i-\frac{1}{2}}\right) = h \sum_{i=0}^n f\left(x_{i-\frac{1}{2}}\right) =$$

$$= 0,2 \cdot (-3,27 - 1,57 + 0,50 + 3,06 + 6,19 + 10,01 + 14,60 + 20,08 + 26,54 + 34,07) = 0,2 \cdot 110,21 = 22,04$$

Метод трапеций:

$$x_i = a + h \cdot i, \text{ где } h = \frac{b-a}{n}, i = 1, 2, \dots, n$$

i	x_i	$f(x)$
0	1,0	-4
1	1,2	-2,46
2	1,4	-0,59
3	1,6	1,71
4	1,8	4,54
5	2,0	8
6	2,2	12,18
7	2,4	17,17
8	2,6	23,07
9	2,8	29,98
10	3	38

$$\int_a^b f(x)dx \approx h \cdot \left(\frac{y_0 + y_n}{2} + \sum_{i=1}^{n-1} y_i \right) =$$

$$= 0,2 \cdot \left(\frac{38 - 4}{2} - 2,46 - 0,59 + 1,71 + 4,54 + 8 + 12,18 + 17,17 + 23,07 + 29,98 \right) \approx 22,12$$

Метод Симпсона:

$$\int_a^b f(x)dx \approx \left(\frac{h}{3} \right) \cdot \left[y_0 + y_n + 4 \cdot (y_1 + y_3 + y_5 + \dots + y_{(n-1)}) + 2 \cdot (y_2 + y_4 + y_6 + \dots + y_{n-2}) \right]$$

$$y_1 + y_3 + y_5 + y_7 + y_9 = -2,46 + 1,71 + 8 + 17,17 + 29,98 = 54,4$$

$$y_2 + y_4 + y_6 + y_8 = -0,59 + 4,54 + 12,18 + 23,07 = 39,2$$

$$\int_a^b f(x)dx \approx \left(\frac{0,2}{3} \right) \cdot (-4 + 38 + 4 \cdot 54,4 + 2 \cdot 39,2) = 22$$

4. Сравнение результатов с точным значением интеграла:

Формула Ньютона – Котеса ($n = 6$):

$$|I_T - I_{\Pi}| = 22 - 22 = 0$$

Формула средних прямоугольников (n = 10):

$$|I_T - I_{\Pi}| = |22,04 - 22| = 0,04$$

Метод трапеций:

$$|I_T - I_{\Pi}| = |22,12 - 22| = 0,12$$

Метод Симпсона:

$$|I_T - I_{\Pi}| = |22 - 22| = 0$$

5. Относительная погрешность вычислений для каждого метода.

$$\delta = \frac{|I_T - I_{\Pi}|}{I_T} \cdot 100\%$$

Формула Ньютона – Котеса (n = 6):

$$\delta = \frac{|I_T - I_{\Pi}|}{I_T} \cdot 100\% = \frac{22 - 22}{22} \cdot 100\% = 0\%$$

Формула средних прямоугольников (n = 10):

$$\delta = \frac{|I_T - I_{\Pi}|}{I_T} \cdot 100\% = \frac{22,04 - 22}{22} \cdot 100\% = 0,18\%$$

Метод трапеций:

$$\delta = \frac{|I_T - I_{\Pi}|}{I_T} \cdot 100\% = \frac{22,14 - 22}{22} \cdot 100\% = 0,55\%$$

Метод Симпсона:

$$\delta = \frac{|I_T - I_{\Pi}|}{I_T} \cdot 100\% = \frac{22 - 22}{22} \cdot 100\% = 0\%$$

Листинг программы:

```
import math
```

```
functions = {
    1: ("2*x**3 - 3*x**2 + 7*x - 10", lambda x: 2*x**3 - 3*x**2 + 7*x - 10),
    2: ("sin(x)", lambda x: math.sin(x)),
    3: ("cos(x)", lambda x: math.cos(x)),
    4: ("x**2", lambda x: x**2),
    5: ("exp(x)", lambda x: math.exp(x)),
    6: ("1/x (пазрыв в 0)", lambda x: 1/x if x != 0 else float('inf')),
    7: ("1/sqrt(x) (пазрыв в 0)", lambda x: 1/math.sqrt(x) if x > 0 else float('inf')),
    8: ("1/(x-2) (пазрыв в 2)", lambda x: 1/(x-2) if x != 2 else float('inf')),
    9: ("1/(x-1)**2 (пазрыв в 1)", lambda x: 1/(x-1)**2 if x != 1 else float('inf'))
}
```

```

methods = {
    1: ("Метод левых прямоугольников", "rectangle_left", 1),
    2: ("Метод правых прямоугольников", "rectangle_right", 1),
    3: ("Метод средних прямоугольников", "rectangle_middle", 2),
    4: ("Метод трапеций", "trapezoidal", 2),
    5: ("Метод Симпсона", "simpson", 4)
}

```

```

class NumericalIntegrator:

```

```

    def __init__(self, func, a, b, eps, n_start=4):
        self.func = func
        self.a = a
        self.b = b
        self.eps = eps
        self.n_start = n_start

```

```

    def rectangle_left(self, n):
        h = (self.b - self.a) / n
        x = self.a
        integral = 0
        for i in range(n):
            val = self.func(x)
            if val == float('inf') or val == float('-inf'):
                return float('inf')
            integral += val
            x += h
        return integral * h

```

```

    def rectangle_right(self, n):
        h = (self.b - self.a) / n
        x = self.a + h
        integral = 0
        for i in range(n):
            val = self.func(x)
            if val == float('inf') or val == float('-inf'):
                return float('inf')
            integral += val
            x += h
        return integral * h

```

```

    def rectangle_middle(self, n):
        h = (self.b - self.a) / n
        x = self.a + h / 2
        integral = 0
        for i in range(n):
            val = self.func(x)
            if val == float('inf') or val == float('-inf'):
                return float('inf')

```

```
    integral += val
    x += h
    return integral * h
```

```
def trapezoidal(self, n):
    h = (self.b - self.a) / n
    left = self.func(self.a)
    right = self.func(self.b)
    if left == float('inf') or left == float('-inf') or right == float('inf') or right == float('-inf'):
        return float('inf')
    integral = (left + right) / 2
    x = self.a + h
    for i in range(1, n):
        val = self.func(x)
        if val == float('inf') or val == float('-inf'):
            return float('inf')
        integral += val
        x += h
    return integral * h
```

```
def simpson(self, n):
    if n % 2 != 0:
        n += 1
    h = (self.b - self.a) / n
    left = self.func(self.a)
    right = self.func(self.b)
    if left == float('inf') or left == float('-inf') or right == float('inf') or right == float('-inf'):
        return float('inf')
    integral = left + right
```

```
    x = self.a + h
    for i in range(1, n, 2):
        val = self.func(x)
        if val == float('inf') or val == float('-inf'):
            return float('inf')
        integral += 4 * val
        x += 2 * h
```

```
    x = self.a + 2 * h
    for i in range(2, n - 1, 2):
        val = self.func(x)
        if val == float('inf') or val == float('-inf'):
            return float('inf')
        integral += 2 * val
        x += 2 * h
```

```
    return integral * h / 3
```

```
def runge_rule(self, method, n, k):
```



```

l_n = method(n)
l_2n = method(2 * n)
if l_n == float('inf') or l_2n == float('inf'):
    return float('inf'), float('inf')
error = abs(l_2n - l_n) / (2 ** k - 1)
return l_2n, error

def compute(self, method, p, max_iter=20):
    history = []
    n = self.n_start

    l_start = method(n)
    if l_start == float('inf'):
        return [], float('inf'), n

    history.append({
        "n": n,
        "value": l_start,
        "error": None
    })

    for iteration in range(max_iter):
        l_2n, error = self.runge_rule(method, n, p)

        if l_2n == float('inf'):
            return history, float('inf'), 2 * n

        history.append({
            "n": 2 * n,
            "value": l_2n,
            "error": error
        })

        if error < self.eps:
            return history, l_2n, 2 * n

        n *= 2
        if n > 2 ** max_iter:
            return history, l_2n, n

    return history, l_2n, n

def find_discontinuity(self):
    for point in [self.a, self.b, (self.a + self.b) / 2]:
        try:
            val = self.func(point)
            if abs(val) > 1e10:
                return point
        except:

```

```

        return point

    for i in range(1, 100):
        point = self.a + i * (self.b - self.a) / 100
        try:
            if abs(self.func(point)) > 1e10:
                return point
        except:
            return point
    return None

def compute_improper(self, method, p, max_iter=20):
    c = self.find_discontinuity()
    if c is None:
        return self.compute(method, p, max_iter)

    original_a, original_b = self.a, self.b
    delta = 1e-7

    if abs(c - self.a) < 1e-12:
        for _ in range(8):
            self.a = original_a + delta
            if self.a >= self.b:
                delta *= 0.5
                continue
            hist, res, n_f = self.compute(method, p, max_iter)
            if res != float('inf'):
                self.a, self.b = original_a, original_b
                return hist, res, n_f
            delta *= 0.1

    self.a, self.b = original_a, original_b
    return [], float('inf'), 0

    if abs(c - self.b) < 1e-12:
        for _ in range(8):
            self.b = original_b - delta
            if self.b <= self.a:
                delta *= 0.5
                continue
            hist, res, n_f = self.compute(method, p, max_iter)
            if res != float('inf'):
                self.a, self.b = original_a, original_b
                return hist, res, n_f
            delta *= 0.1
    self.a, self.b = original_a, original_b
    return [], float('inf'), 0

```

```

for _ in range(8):
    left_b = c - delta
    right_a = c + delta
    if left_b <= self.a or right_a >= self.b:
        delta *= 0.5
        continue

total_hist = []
total_res = 0
ok = True

self.a, self.b = original_a, left_b
hist1, res1, _ = self.compute(method, p, max_iter)
if res1 != float('inf'):
    total_res += res1
    total_hist.extend(hist1)
else:
    ok = False

self.a, self.b = right_a, original_b
hist2, res2, _ = self.compute(method, p, max_iter)
if res2 != float('inf'):
    total_res += res2
    total_hist.extend(hist2)
else:
    ok = False

self.a, self.b = original_a, original_b

if ok and total_res != float('inf'):
    return total_hist, total_res, max(len(hist1), len(hist2)) * 2

delta *= 0.5

return [], float('inf'), 0

def main():
    print("Численное интегрирование")

    print("\nДоступные функции:")
    for key, (desc, _) in functions.items():
        print(f" {key}. {desc}")

    while True:
        try:
            func_choice = int(input("\nВыберите функцию (1-9): "))
            if func_choice in functions:

```

```

        break
    print("Ошибка: введите число от 1 до 9")
except ValueError:
    print("Ошибка: введите целое число")

func_desc, func = functions[func_choice]
print(f"Выбрана функция: f(x) = {func_desc}")

while True:
    try:
        a = float(input("\nВведите нижний предел a: "))
        b = float(input("Введите верхний предел b: "))
        if a < b:
            break
        print("Ошибка: a должно быть меньше b")
    except ValueError:
        print("Ошибка: введите число")

while True:
    try:
        eps = float(input("\nВведите точность вычисления (например, 0.001): "))
        if eps > 0:
            break
        print("Ошибка: точность должна быть положительной")
    except ValueError:
        print("Ошибка: введите число")

print("\nДоступные методы:")
for key, (name, _, _) in methods.items():
    print(f" {key}. {name}")

while True:
    try:
        method_choice = int(input("\nВыберите метод (1-5): "))
        if 1 <= method_choice <= 5:
            break
        print("Ошибка: введите число от 1 до 5")
    except ValueError:
        print("Ошибка: введите целое число")

integrator = NumericalIntegrator(func, a, b, eps)

print("Результаты:")

name, method_str, p = methods[method_choice]
method = getattr(integrator, method_str)

c = integrator.find_discontinuity()
if c is not None:

```

```

if abs(c - a) < 1e-12 and "1/x" in func_desc and "sqrt" not in func_desc:
    print("Интеграл не существует (расходится)")
    return
if a < c < b and ("1/(x-2)" in func_desc or "1/(x-1)**2" in func_desc):
    print("Интеграл не существует (расходится)")
    return

history, result, n_final = integrator.compute_improper(method, p)

if result == float('inf'):
    print("Интеграл не существует")
else:
    print(f"Начальное n = {integrator.n_start}")

    for step in history:
        if step["error"] is None:
            print(f" n = {step['n']:4d} | Значение = {step['value']:.8f} | Погрешность = (нет
данных)")
        else:
            print(f" n = {step['n']:4d} | Значение = {step['value']:.8f} | Погрешность =
{step['error']:.5f}")

    print(f"\nТребуемая точность {eps} достигнута при n = {n_final}")
    print(f"Значение интеграла: {result:.8f}")

if __name__ == "__main__":
    main()

```

Вывод:

В данной лабораторной работе я изучил численные методы интегрирования.